

# Домашнее задание 7

Лиза Фоменко

## Задание 1

*Реализуйте очередь через два стека. Оцените асимптотику операций с получившейся очередью.*

Возьмем два стека. Дальше алгоритм такой:

```
class Queue:
    def __init__(self, data):
        self.stack1 = Stack()          # это стек1, в него пушим
        self.stack2 = Stack()          # это стек2, из него выдаем объекты

    def push(self, data):
        # когда кладем объекты в очередь, пушим их в стек1 за O(1)
        for value in data:
            self.stack1.push(x)

    def pop(self):

        # выдаем всегда тот объект, который находится на пуле второго стека

        # если второй стек пуст, а первый - нет, то:
        if self.stack2.isempty():
            while not self.stack1.isempty():

                # перекладываем объекты из стек1 в стек2 (стек1_пул - стек2_пуш)
                # так те объекты, которые пришли раньше, окажутся "снизу" второго стека
                self.stack2.push(self.stack1.pop())

            # просто пул второго стека - достаем элемент
            result = self.stack2.pop()

        else:
            # если во втором стеке есть элементы, достаем верхний
            result = self.stack2.pop()
        return result
```

Более словесный формат описания: возьмем два стека - обозначим их как стек1 и стек2. Если в нашу очередь приходит элемент, то "положить его в очередь" - "запустить в стек1". Эта операция выполняется за  $O(1)$ .

Если хотим сделать шаг dequeue, то есть достать первый из добавленных элементов, то делаем пул из стек2: "dequeue из очереди" - "пул из стека2".

Кладем исходные объекты в стек1. Сейчас мы можем доставать их в порядке, обратному приходу в очередь. Поэтому когда нам необходимо достать элемент из очереди, нам нужно все элементы достать из стек1 (пул) и запустить их в стек2. С помощью такой операции мы будем доставать объекты из стека2 в порядке, необходимом для очереди (элемент пришедший первым будет "сверху" стека2).

Оценим асимптотику: enqueue = push в очередь - это обычный push в стек1, который осуществляем за  $O(1)$

Дополнительная оценка на dequeue = pull из очереди = pull из стека2. Если стек2 не пустой, то эта операция выполняется за  $O(1)$  (pull из стека обычный).

А если стек2 пустой? Тогда нам необходимо переложить все элементы из стека1 в стек2. Но каждое перекладывание элемента стоит  $O(1)$  (одна операция pull из стека1 и push в стек2). Для  $N$  элементов полная асимптотика перекладывания  $O(N)$ .

НО: мы можем рассуждать так: если всего за перекладывание мы сделали  $O(N)$  операций для  $N$  элементов, значит, можно амортизационно рассчитать, сколько операций приходится на один элемент -  $O(N)/N = O(1)$ . То есть среди всех доп действий, на один элемент все еще приходится  $O(1)$  операция. Так как каждый объект лишь единожды перекладывается из стек1 в стек2, мы можем считать, что стоимость операции pull для него - это  $\text{pull\_stack1} + \text{push\_stack2} + \text{pull\_stack2} = O(1)$ . В общем, в какой-то момент времени мы "зависаем" и действительно достаем нужный элемент за  $O(N)$  временных шагов, но амортизационно каждый объект все еще достается за  $O(1)$ .

## Задание 2

*Найдите значение переменной  $y$  после выполнения следующего алгоритма.  $N$  можно округлить до степени двойки.  $k(x)$  - число подряд идущих нулей, начиная с младшего бита бинарной записи  $x$ .*

$x, y = 0, 0 \#n\text{-bit}$

```
for i in range(N):
    x += 1
    y += k(x)
```

Давайте рассмотрим последовательность  $x$ .

01, 10, 11, 100, 101, 110, 111, 1000, ... Что значит наличие 0 в конце числа? - оно нацело делится на 2 (сдвиг на 1 бит - остатка не будет)

Что значит наличие 00 в конце числа? - оно нацело делится на  $2^2$  (то есть возможен сдвиг на 2 бита - остатка не будет)

Аналогичным образом, наличие  $n$  0 подряд в числе (с меньшего разряда) означает, что число делится на  $2^n$  без остатка.

Далее рассмотрим следующее:

| колво 0 | делится на | сколько   |
|---------|------------|-----------|
| 1       | $2^1$      | $2^{n-1}$ |
| 2       | $2^2$      | $2^{n-2}$ |
| 3       | $2^3$      | $2^{n-3}$ |
| ...     | ...        | ...       |
| 4       | $2^4$      | $2^{n-4}$ |
| $n-2$   | $2^{n-2}$  | $2^2$     |
| $n-1$   | $2^{n-1}$  | 2         |
| $n$     | $2^n$      | 1         |

Заметим, что, если число делится на  $2^i$ , то оно делится на  $2^k$  для всех  $k < i$ . Если мы просто просуммируем по столбцу "сколько" \* "кол-во то ответ будет неверным - мы много раз учтем одни и те же нули. Вместо этого пойдем следующее:

число  $x$ , делящееся на  $2^i$ , учитывается по 1 разу в каждой группе на делимость на  $2^k$  для всех  $k < i$ . Тогда количество нулей в этом числе ни что иное как количество таких  $k \leq i$ . Раз число входит по 1 разу в каждую группу, мы можем просто сложить по ряду "сколько" размазав все нули числа по разрядам (соотв группе). Суммируем:

$$2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 4 + 2 + 1 = \sum_{i=0}^{n-1} 2^i = 2^n - 1$$

.

Теперь, так как мы предполагали, что  $N = 2^n$ , подставим:

$$y = N - 1$$

## Задание 3

**Постройте алгоритм, принимающий на вход массив  $a_1, \dots, a_n$  и позволяющий находить значения сумм вида  $(a_{m+1} - a_m)^2 + \dots + (a_k - a_{k-1})^2$  за  $O(1)$ . Оцените необходимый объем дополнительной памяти.**

Кажется, что это базовая задача на префиксные суммы

Заметим, что  $(a_{m+1} - a_m)^2 + \dots + (a_k - a_{k-1})^2 = \sum_{i=1}^k (a_i - a_{i-1})^2 - \sum_{j=1}^m (a_j - a_{j-1})^2$ . Тогда, если мы храним все суммы вида  $\sum_{i=1}^l (a_i - a_{i-1})^2$  для всех  $1 \leq l \leq n$ , то запрашиваемая операция будет производиться за  $O(1)$  (просто два доступа к элементу и их вычитание)

Далее заметим следующее:  $\sum_{i=1}^k (a_i - a_{i-1})^2 = \sum_{i=1}^{k-1} (a_i - a_{i-1})^2 + (a_k - a_{k-1})^2$  для любых  $k$ . Зная сумму для  $k-1$ , можем за один шаг посчитать сумму для  $k$ . То есть итеративно будем ее считать по следующему алгоритму:

```
new_arr = [0]
for i in range(1, n):
    # N-1 раз
    prev_sum = new_arr[-1]          # последняя вычисленная сумма
    new_sum = prev_sum + (arr[i] - arr[i-1])**2 # за O(1)
    new_sum.append(new_sum)
```

Теперь у нас есть массив `new_arr` длины  $n$  (так как 0 в начале добавлен, можно и убрать, но все равно) - доп память  $O(n)$ .  $k$  элемент массива равен  $\sum_{i=1}^k (a_i - a_{i-1})^2$ .

Тогда для данных  $k, m$  запрашиваемое выражение - это просто `new_arr[k] - new_arr[m]`. За  $O(1)$ .

## Задание 4

**Ваш лектор по алгоритмам нашёл два одинаковых шарика из неизвестного материала и внезапно решил измерить их прочность в этажах  $n$ -этажного небоскрёба. Прочность равна номеру минимального этажа, при броске шарика из окна которого шарик разобьётся (максимум 100). Считаем, что если шарик уцелел, то его прочность после броска не уменьшилась. Сколько бросков шариков необходимо и достаточно для нахождения прочности?**

"Необходимо и достаточно оцениваем для худшего случая.

Если бы был 1 шарик - нам бы пришлось кидать шарик с каждого этажа (даже если киджали бы с каждого 2го, то, если шарик на этаже  $k$  уцелел, а на этаже  $k+2$  разбился, то мы не можем установить его прочность между  $k+1$  и  $k+2$  этажами).

В лучшем случае, когда у нас было бы неограничено шариков, задача свелась бы к обычному бинарному поиску (за  $\lceil \log_2 100 \rceil = 7$  шагов).

У нас два шарика. Заметим, что, если мы кинули шарик1 с  $k_i$  (и он выжил), а затем с  $k_{i+1}$  этажа (и он умер), то второй шарик придется кидать подряд с  $k_i+1$  этажа до  $k_{i+1}-1$  этажа, а это  $k_{i+1} - k_i - 1$  бросок.

Давайте составим последовательность бросков первого шара:  $k_1, k_2, \dots, k_m$ , где  $k_i$  - это какой-то этаж (не ий в общем понимании, очевидно).

Тогда максимально возможное количество бросков второго шара между каждыми двумя этажами составит:  $k_2 - k_1 - 1, k_3 - k_2 - 1, \dots, k_m - k_{m-1} - 1$ .

Давайте для всех  $i$  оценим количество шагов:

$i + k_i - k_{i-1} - 1$ , где  $i$  - количество бросков шара1 (с этажа  $k_1, k_2, \dots, k_i$  соответственно), а  $k_i - k_{i-1} - 1$  - максимально возможное количество бросков шара2 после этого.

Интуиция: если у нас этажи будут распределены равномерно, то с ростом прочности шарика, нам придется делать все БОльшее количество бросков. Это плохо, так как оценка завышена.

Значит, этажи не должны быть распределены равномерно (скорее наоборот, расстояние между ними со временем должно уменьшаться).

Попробуем распределить этажи: пусть необходимое и достаточное число шагов равно  $d$ . Мы рассматриваем худший случай, когда шар2 делает максимальное количество шагов.

1.  $i = 1$ : шар1 сделал 1 ход с  $k_1$  этажа. Тогда второй шар может сделать  $d-1$  ход, что не меньше  $k_1-1$  (с 1 по  $k_1-1$  этажи)  $\Rightarrow k_1 \leq d$
2.  $i = 2$ : шар1 сделал 2 хода с  $k_1$  и  $k_2$  этажей. Тогда второй шар может сделать  $d-2$  ходов, что не меньше  $k_2 - k_1 - 1$  (с  $k_1+1$  по  $k_2-1$  этажи)  $\Rightarrow k_2 - k_1 - 1 \leq d-2$
3.  $i = 3$ : шар1 сделал 3 хода с  $k_1, k_2, k_3$  этажей. Тогда второй шар может сделать  $d-3$  ходов, что не меньше  $k_3 - k_2 - 1$  (с  $k_2+1$  по  $k_3-1$  этажи)  $\Rightarrow k_3 - k_2 - 1 \leq d-3$

4.  $i = 4$ : шар1 сделал 4 хода с  $k_1, k_2, k_3, k_4$  этажей. Тогда второй шар может сделать  $d - 4$  ходов, что не меньше  $k_4 - k_3 - 1$  (с  $k_3 + 1$  по  $k_4 - 1$  этажи)  $\Rightarrow k_4 - k_3 - 1 \leq d - 4$

Получили систему неравенств:  $k_1 \leq d$

$$k_2 - k_1 \leq d - 1$$

$$k_3 - k_2 \leq d - 2$$

$$k_4 - k_3 \leq d - 3$$

$$k_{i+1} - k_i \leq d - i$$

Для оценки заменим все  $\leq$  на  $=$ .

$$k_1 = d$$

$$k_2 - k_1 = d - 1 \Rightarrow k_2 = k_1 + (d - 1) = d + (d - 1)$$

$$k_3 - k_2 = d - 2 \Rightarrow k_3 = k_2 + (d - 2) = d + (d - 1) + (d - 2)$$

$$k_4 - k_3 = d - 3 \Rightarrow k_4 = k_3 + (d - 3) = d + (d - 1) + (d - 2) + (d - 3)$$

$$k_{i+1} - k_i = d - i = d + (d - 1) + (d - 2) + \dots + (d - i)$$

$$k_d = k_{d-1} = d + (d - 1) + \dots + 1 = \frac{d(d+1)}{2}$$

Также мы знаем, что максимальное количество этажей - 100, поэтому посчитаем  $k_d = \frac{d(d+1)}{2} = 100$

Отсюда  $d = 13.65 \Rightarrow$  округляем до целого  $\Rightarrow d = 14$

ответ:  $d = 14$